Cap. 4: Regresión por Mínimos Cuadrados

Optimización Numérica — Maestría en Matemática

Luis Berlioz

7 de noviembre de 2026

Temas del capítulo

Mínimos cuadrados lineales

La descomposición en valores singulares

Norma mínima y compressed sensing

Conditioning y regularización

Restricciones y programación cuadrática

Mínimos cuadrados no lineales

El método del gradiente conjugado

Cierre

El problema en una línea

Un sistema lineal **con más ecuaciones que incógnitas** no tiene solución exacta. Se convierte en un problema de optimización:

$$Ax = b \qquad (4.1) \qquad \Longrightarrow \qquad \min_{x} \ \|Ax - b\|_2^2 \qquad (4.2)$$

- La solución es **analítica y cerrada**:

$$x = A^{\dagger} b \qquad (4.3)$$

  con $A^{\dagger}$ la **pseudo-inversa de Moore–Penrose**.
- La generalización no lineal $\min_{\theta} \|N_\theta(x) - y\|_2^2$ **no es convexa** y exige métodos iterativos (§4.7).

La cadena del capítulo

solución cerrada → SVD → norma mínima → conditioning → regularización → restricciones y QP → Gauss–Newton → gradiente conjugado

Cuadrado, sobredeterminado, subdeterminado

- **Cuadrado** ($m = n$): invertir la matriz; solución exacta.
- **Sobredeterminado** ($m \gg n$): más ecuaciones que incógnitas. *Tall-skinny*. Casi nunca hay solución exacta $\Rightarrow$ se minimiza el error.
- **Subdeterminado** ($m \ll n$): menos ecuaciones que incógnitas. *Short-fat*. Infinitas soluciones $\Rightarrow$ se elige la de norma mínima.

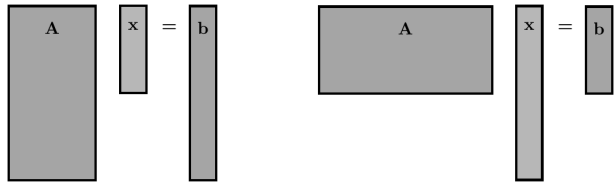


Fig. 4.2: regresión sobredeterminada (izq.) y subdeterminada (der.)

Brunton, *Optimization Bootcamp*, Cap. 4

El modelo de regresión lineal

Los datos se ordenan como filas de una matriz $A \in \mathbb{R}^{m \times n}$ y un vector de resultados $b \in \mathbb{R}^m$. Se busca el peso $x \in \mathbb{R}^n$ tal que b se aproxime por combinación lineal de las **columnas** de A:

$$Ax \approx b$$

- Cada columna de A es un **atributo** o predictor; cada fila, una **medición**.
- b casi nunca es exacto: viene contaminado con ruido de medición, modelado como ruido blanco Gaussiano i.i.d.

$$b = b_{\text{true}} + \epsilon$$

- Motivación física (p. 122): el equilibrio de fuerzas en las juntas de un puente da una ecuación por junta. Con ruido en b o imperfecciones en A, resolver por las fuerzas internas es un **problema inverso de mínimos cuadrados**.

Brunton, *Optimization Bootcamp*, Cap. 4

Las ecuaciones normales

Expandir el objetivo y anular el gradiente:

$$\begin{aligned} \|Ax - b\|_2^2 &= (Ax - b)^T (Ax - b) \\ &= x^T A^T A x - x^T A^T b - b^T A x + b^T b \end{aligned} \tag{4.5c}$$

$$\nabla_x \|Ax - b\|_2^2 = 2A^T A x - 2A^T b = 0 \implies \boxed{A^T A x = A^T b} \tag{4.6b}$$

$$x = (A^T A)^{-1} A^T b \tag{4.6c}$$

La trampa

(4.6c) **solo vale si** $A^T A$ **es invertible**, es decir si A tiene n columnas linealmente independientes y $m \geq n$. Es la fórmula del caso **sobredeterminado**; no sirve para *short-fat*.

Brunton, *Optimization Bootcamp*, Cap. 4

La pseudo-inversa de Moore–Penrose

$$A^\dagger = (A^T A)^{-1} A^T \qquad \text{(izquierda)} \qquad\qquad A^\dagger = A^T (A A^T)^{-1} \qquad \text{(derecha)}$$

- Está definida incluso para A no cuadrada, pero cada forma exige rango completo en su lado.
- Numéricamente **no se invierte** $A^T A$: su número de condición es el cuadrado del de A. Se usa la **SVD**:

$$A = U \Sigma V^T \quad \Longrightarrow \quad A^\dagger = V \Sigma^{-1} U^T \quad \Longrightarrow \quad x = V \Sigma^{-1} U^T b \tag{4.7}$$

- U y V son ortogonales: invertirlas es transponerlas. Σ es diagonal: invertirla son n divisiones.

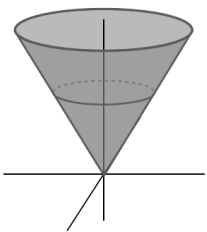
Por qué importa

La SVD es la técnica de elección para invertir matrices grandes: es numéricamente estable, siempre existe y no necesita formar $A^T A$.

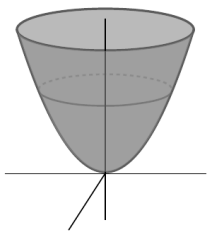
Brunton, *Optimization Bootcamp*, Cap. 4

Norma 2 frente a norma 2 al cuadrado

$\|\mathbf{x}\|_2 = \sqrt{x_1^2 + x_2^2}$



$\|\mathbf{x}\|_2^2 = x_1^2 + x_2^2$



- Izquierda: $\|x\|_2 = \sqrt{x_1^2 + x_2^2}$. Sus conjuntos de nivel son **círculos**.
- Derecha: $\|x\|_2^2 = x_1^2 + x_2^2$. Mismos conjuntos de nivel, misma solución óptima.
- Minimizar $\|\cdot\|_2$ o $\|\cdot\|_2^2$ da **el mismo** x^*: el cuadrado es monótono.
- Pero el cuadrado **pesa distinto** los errores: un error grande cuesta mucho más que dos pequeños. De ahí la sensibilidad a outliers.

Brunton, *Optimization Bootcamp*, Cap. 4

Ejemplo: datos aleatorios (E4.1)

Pseudo-inversa calculada por SVD frente a `np.linalg.pinv`:

```
import numpy as np
np.random.seed(42)
m, n = 20, 5
A = np.random.randn(m, n)
b = np.random.randn(m)

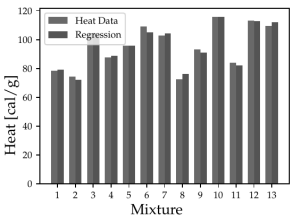
U, S, VT = np.linalg.svd(A, full_matrices=0)
x = VT.T @ np.linalg.inv(np.diag(S)) @ U.T @ b

x = np.linalg.pinv(A) @ b          # alternativa
```

- `full_matrices=0` encuentra la descomposición **SVD económica**: solo los valores singulares no nulos, matrices $m \times n$ en vez de $m \times m$.
- Ambas líneas dan el mismo x hasta error de redondeo.

Brunton, *Optimization Bootcamp*, Cap. 4

Ejemplo: calor del cemento (E4.2)



	Ingredient	x_j
1	Tricalcium aluminate	2.1930
2	Tricalcium silicate	1.1533
3	Tetracalcium alumiferrite	0.7585
4	Beta-dicalcium silicate	0.4863

Fig. 4.5: calor generado por 13 mezclas de cemento.

$A \in \mathbb{R}^{13\times 4}$: cuatro ingredientes, trece mezclas. Se buscan los pesos x que relacionan proporciones con calor.

Ingrediente	x_j
Tricalcium aluminate	2.1930
Tricalcium silicate	1.1533
Tetracalcium alumifer.	0.7585
Beta-dicalcium silicate	0.4863

Es el experimento clásico de Hald (1952): cuatro predictores fuertemente correlacionados y solo trece datos.

Brunton, *Optimization Bootcamp*, Cap. 4

Ejemplo: precios de vivienda en Boston (E4.3)

- 506 viviendas, 13 atributos: criminalidad per cápita, tasa de impuesto predial, etc.
- $A \in \mathbb{R}^{506\times 13}$: problema claramente sobredeterminado.
- **Hay que añadir una columna de unos**: da el término constante (la ordenada al origen). Equivale a centrar cada columna restando su media.
- El ajuste no captura los precios extremos altos: los **modelos lineales simples fallan en las colas**.

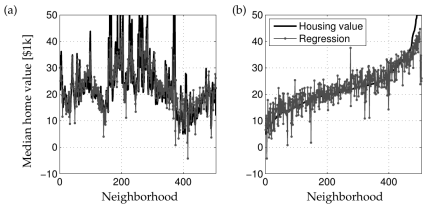


Fig. 4.6: valor real frente a valor ajustado.

Brunton, *Optimization Bootcamp*, Cap. 4

Outliers: por qué ℓ_2 no basta

- Un outlier entra **al cuadrado**: su error domina y *arrastra* toda la recta.
- Alternativa robusta: **desviación absoluta mínima**

$$\min_x \ \|Ax - b\|_1 \tag{4.8}$$

- Regla general: usar ℓ_1 como pérdida suele ser más robusto a outliers. El precio es perder la solución cerrada: hay que resolver un problema de optimización.

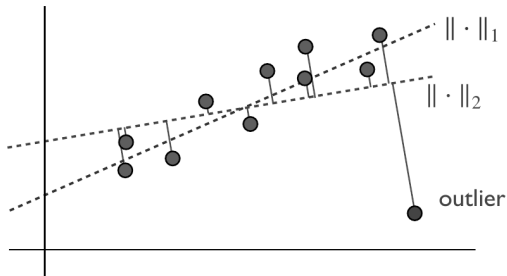


Fig. 4.7: ℓ_2 (sensible) frente a ℓ_1 (robusta).

Brunton, *Optimization Bootcamp*, Cap. 4

Panel 4.1: ¿mínimos cuadrados o desviación absoluta?

Doscientos años de debate, con una lectura estadística limpia:

	Mínimos cuadrados (ℓ_2)	**LAD** (ℓ_1)
Estima	la **media**	la **mediana**
Robustez a outliers	baja	alta
Óptimo	cerrado (ecuaciones normales)	programación lineal
Historia	Gauss y Legendre, $\sim$1800	Boscovich, 1757

- La media es insesgada para ruido Gaussiano; la mediana es insesgada para ruido Laplaciano.
- Ejemplo del libro (sep. 2024): riqueza mediana en EE. UU. $\sim$ \$200k frente a riqueza media $\sim$ \$1,000k. **La media la mueven los outliers; la mediana no.**

Brunton, *Optimization Bootcamp*, Cap. 4

Los cuatro subespacios fundamentales

- $\mathrm{col}(A)$: span de las columnas (*rango*).
- $\mathrm{row}(A) = \mathrm{col}(A^T)$: span de las filas.
- $\ker(A)$: vectores con $Ax = 0$.
- $\ker(A^T)$: complemento ortogonal de $\mathrm{col}(A)$.

Descomposición en suma directa (p. 131)

$$\mathrm{col}(A) \oplus \ker(A^T) = \mathbb{R}^m \tag{4.9a}$$

$$\mathrm{col}(A^T) \oplus \ker(A) = \mathbb{R}^n \tag{4.9b}$$

Consecuencias sobre $Ax = b$:

- $b \in \mathrm{col}(A)$ y $\dim \ker(A) \neq 0 \Rightarrow$ **infinitas soluciones** (garantizado si A es short-fat).
- $b \notin \mathrm{col}(A) \Rightarrow$ **ninguna solución**: el sistema es inconsistente.
- Se elige entonces **la mejor aproximación** (*least squares*) o **la de norma mínima**, según el régimen.

Brunton, *Optimization Bootcamp*, Cap. 4

Las dos soluciones canónicas

Sobredeterminado: mínimos cuadrados

$$\min_x \ \|Ax - b\|_2^2$$

$$x = A^\dagger b = (A^T A)^{-1} A^T b$$

Minimiza el error; b se proyecta sobre $\mathrm{col}(A)$.

Subdeterminado: norma mínima

$$\min_x \ \|x\|_2^2 \quad \text{s.a. } Ax = b$$

$$x = A^T (AA^T)^{-1} b$$

Satisface la ecuación exacta con el x más corto posible.

- Ambas son la **misma** pseudo-inversa $A^\dagger$, aplicada por lados distintos.
- Obs. 1 (p. 131): con matrices **aleatorias** los estereotipos son ciertos con altísima probabilidad. Para A aleatoria con $m \gg n$ es casi imposible que $b \in \mathrm{col}(A)$.

Brunton, *Optimization Bootcamp*, Cap. 4

Qué es la SVD

$$A = U\Sigma V^T \tag{4.10}$$

- $U \in \mathbb{R}^{m\times m}$ y $V \in \mathbb{R}^{n\times n}$ **ortogonales**: columnas ortonormales.
- Columnas de U: **vectores singulares izquierdos**.
- Columnas de V: **derechos**.
- Diagonal de Σ: valores singulares ordenados

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_n \geq 0.$$

- El **rango** de A es el número de σ_j no nulos.
- Para matrices complejas, U, V son unitarias y T pasa a ser transpuesta conjugada $*$.

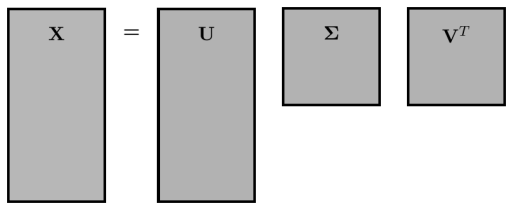


Fig. 4.8: la SVD descompone en filas y columnas ortogonales ordenadas por varianza.

Brunton, *Optimization Bootcamp*, Cap. 4

SVD económica y truncada

SVD económica (solo σ no nulos, $m \geq n$):

$$A = \hat{U}\hat{\Sigma}V^T \tag{4.11}$$

SVD truncada de rango r:

$$\tilde{A} = \tilde{U}\tilde{\Sigma}\tilde{V}^T$$

$\tilde{U}, \tilde{V}$: primeras r columnas; $\tilde{\Sigma}$: bloque $r \times r$.

- "Truncar" = quedarse con los r patrones dominantes.
- Es el mismo mecanismo que **PCA**.

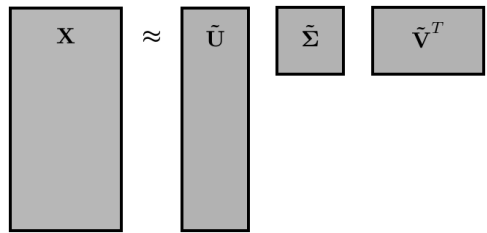


Fig. 4.9: aproximación truncada con r valores singulares.

Brunton, *Optimization Bootcamp*, Cap. 4

La SVD como suma de matrices de rango 1

Panel 4.2 (p. 133)

Como Σ es diagonal, la SVD se lee como una suma de términos $\sigma_j u_j v_j^T$, cada uno de rango 1 y ortogonal a los demás:

$$X \approx \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \cdots + \sigma_r u_r v_r^T$$

Ordenados por magnitud, los primeros términos capturan lo grueso; los últimos, el detalle fino (y el ruido).

Subespacios en términos de U, V:

- $\mathrm{col}(A) = \mathrm{col}(\hat{U})$
- Complemento ortogonal de $\mathrm{col}(A)$: $\mathrm{col}(\hat{U}^{\perp})$
- $\mathrm{row}(A)$ generado por columnas de V
- $\ker(A) = \mathrm{col}(V^{\perp})$

Por qué es la herramienta

Da a la vez: rango numérico, base de los cuatro subespacios, pseudo-inversa estable, aproximación óptima de bajo rango y diagnóstico de conditioning. Todo con una sola factorización.

Brunton, *Optimization Bootcamp*, Cap. 4

Resolver $Ax = b$ con la SVD

Sustituyendo la SVD truncada exacta ($A = \hat{U}\hat{\Sigma}V^T$) y "invirtiendo" cada factor:

$$A^\dagger \triangleq V\hat{\Sigma}^{-1}\hat{U}^T \qquad \Longrightarrow \qquad A^\dagger A = VV^T \tag{4.12}$$

$$A^\dagger A x^* = A^\dagger b \implies x^* = V\hat{\Sigma}^{-1}\hat{U}^T b \tag{4.13}$$

- Sustituyendo de vuelta: $Ax^* = \hat{U}\hat{U}^T b$ (4.14b).
- **Interpretación:** x^* **es la proyección de** b **sobre** $\mathrm{col}(\hat{U})$. El residuo $b - \hat{U}\hat{U}^T b$ es la parte de b que vive en $\ker(A^T)$ y que *ningún* x puede explicar.
- $A^\dagger A = I$ solo si la SVD captura *todos* los valores singulares no nulos. Con rango r se obtiene $\tilde{V}\tilde{V}^T$: una aproximación de la identidad.

Brunton, *Optimization Bootcamp*, Cap. 4

Teorema de Eckart–Young

Teorema 1 (Eckart–Young [159], p. 135)

La mejor aproximación de rango r en norma de Frobenius es la SVD truncada:

$$\underset{\tilde{X}:\ \mathrm{rank}(\tilde{X})=r}{\arg\min} \|X - \tilde{X}\|_F = \tilde{U}\tilde{\Sigma}\tilde{V}^T \tag{4.15}$$

con $\|X\|_F = \left(\sum_{ij} X_{ij}^2\right)^{1/2}$, equivalente a la norma 2 de $X(:)$.

Error exacto en Frobenius:

$$\|X - \tilde{X}\|_F^2 = \sum_{k=r+1}^{m} \sigma_k^2 \tag{4.16}$$

Toda matriz de rango r tiene al menos este error. El error relativo (4.17) mide la **varianza faltante** — la métrica que usa PCA.

Sorprendentemente, también en norma espectral:

$$\|X - \tilde{X}\|_2 = \sigma_{r+1} \tag{4.19}$$

porque $X - \tilde{X} = \sum_{k>r} \sigma_k u_k v_k^T$ y el máximo se alcanza en $v = v_{r+1}$.

Schmidt (Gram–Schmidt) ya lo había probado para espacios de funciones; Eckart y Young lo redescubrieron en 1936.

Brunton, *Optimization Bootcamp*, Cap. 4

Costo de la SVD

Algoritmo (p. 134):

1. Reducir A a **bidiagonal**. Con $m \gg n$: primero factorización **QR**, luego reflejos de Householder.
2. SVD de la bidiagonal con el **QR modificado de Golub–Kahan**.

Entonces $A^{\dagger}$ es barata:

- invertir U o V: multiplicar por la transpuesta, $O(m^2)$;
- invertir Σ: $O(m)$, es diagonal;
- *contraste:* invertir una matriz densa cuesta $O(m^3)$.

Regla práctica

Nunca se forma ni se invierte $A^T A$: se factoriza A una vez y se reutiliza la misma SVD para mínimos cuadrados, norma mínima, regularización y truncamiento.

Brunton, *Optimization Bootcamp*, Cap. 4

El problema de norma mínima

Caso **subdeterminado**: hay infinitas soluciones de $Ax = b$. Se elige la más corta:

$$\min_x \ \|x\|_2^2 \qquad \text{s.a. } Ax = b \tag{4.22a–b}$$

- Aquí el objetivo es $\|x\|_2^2$, **no** el residual. La ecuación entra como **restricción**.
- Geométricamente: la bola ℓ_2 de radio mínimo que toca el hiperplano $Ax = b$.
- La solución ℓ_2 mínima es **densa**: reparte el peso entre todas las coordenadas.

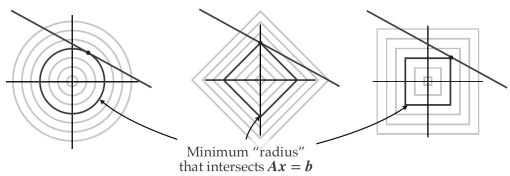


Fig. 4.11: las bolas ℓ_2, ℓ_1 y ℓ_∞ cortando la recta $Ax = b$.

Brunton, *Optimization Bootcamp*, Cap. 4

Derivación por multiplicadores de Lagrange

$$L(x, \lambda) = x^T x + \lambda^T (Ax - b) \tag{4.23}$$

$$\nabla_x L = 2x + A^T \lambda = 0 \implies x = -\tfrac{1}{2} A^T \lambda \tag{4.24}$$

$$\nabla_\lambda L = Ax - b = 0 \implies -AA^T \lambda = b \implies \lambda = -2(AA^T)^{-1} b \tag{4.25}$$

$$\boxed{x = A^T (AA^T)^{-1} b} \tag{4.26–4.27}$$

Pseudo-inversa derecha

Requiere que AA^T sea invertible, es decir que A tenga **rango de fila completo** (lo genérico en una matriz short-fat). Con la SVD es la misma fórmula $x = V\Sigma^{-1} U^T b$: $A^\dagger$ es *una sola* matriz.

Brunton, *Optimization Bootcamp*, Cap. 4

Compressed sensing: la idea

- Imágenes y audio son **compresibles**: en una base Fourier o wavelet, casi todos los coeficientes son despreciables. Es la base de JPEG y MP3.
- Pero el paradigma clásico **mide todo y comprime después**.
- Invertirlo: medir solo una **submuestra aleatoria** y reconstruir los pocos modos activos. Del millón de píxeles de una imagen, potencialmente basta el 10 %.

Formulación

Señal compresible $x = \Psi s$, con s K-**disperso** (K entradas no nulas) y mediciones $y = Cx$:

$$y = Cx = C\Psi s = \Theta s \tag{4.29}$$

con $C \in \mathbb{R}^{p \times n}$ y $p \ll n$: sistema subdeterminado.

Brunton, *Optimization Bootcamp*, Cap. 4

De ℓ_0 a ℓ_1: la relajación convexa

La solución más dispersa es la de **cardinalidad mínima**:

$$\hat{s} = \arg\min_{s} \|s\|_0 \qquad \text{s.a. } y = C\Psi s \tag{4.30}$$

- $\|\cdot\|_0$ cuenta entradas no nulas: **no convexa** y combinatoria. Exige verificar todos los K-dispersos posibles $\Rightarrow$ **intratable** para n y K moderados.
- Bajo condiciones sobre la matriz de medición C, se relaja a un problema **convexo**:

$$\hat{s} = \arg\min_{s} \|s\|_1 \qquad \text{s.a. } y = C\Psi s \tag{4.31}$$

- **La solución ℓ_1 mínima es dispersa; la ℓ_2 mínima no lo es.** La bola ℓ_1 tiene vértices en los ejes, y el óptimo casi siempre cae en uno.
- Con ruido, $\|C\Psi s - y\|_2 < \epsilon$ (4.32) o la forma penalizada $\|C\Psi s - y\|_2 + \lambda\|s\|_1$ (4.33).

Brunton, *Optimization Bootcamp*, Cap. 4

Número de condición: el ruido amplificado

$$\kappa(A) = \frac{\sigma_{\text{máx}}(A)}{\sigma_{\text{mín}}(A)} \tag{4.34}$$

Mide la sensibilidad de A frente a errores en la entrada. Si b viene con error ϵ_b:

$$A(x + \epsilon_x) = b + \epsilon_b \qquad \Longrightarrow \qquad \frac{\|x\|}{\|\epsilon_x\|} = \frac{\sigma_{\text{mín}}}{\sigma_{\text{máx}}} \frac{\|b\|}{\|\epsilon_b\|}$$

Regla de oro

El ruido en b se amplifica por $\kappa(A)$ al invertir. Con columnas casi paralelas, $\kappa(A)$ es enorme y $\kappa(A^T A) = \kappa(A)^2$ es peor: por eso **jamás** se resuelve por ecuaciones normales en ese régimen.

Mitigación por truncamiento (p. 140): descartar los σ bajo $\kappa_{\text{obj}} \cdot \sigma_{\text{máx}}$ eleva el $\sigma_{\text{mín}}$ efectivo; el precio es un subespacio $\hat{U}$ más pequeño.

Brunton, *Optimization Bootcamp*, Cap. 4

Regularización de Tikhonov (ridge)

Añadir una penalización sobre $\|x\|_2^2$:

$$x = \arg\min_{x'} \ \|Ax' - b\|_2^2 + \alpha\|x'\|_2^2 \tag{4.38}$$

$$x = (A^T A + \alpha I)^{-1} A^T b \tag{4.39}$$

- El término αI **desplaza los valores propios lejos de cero**: el número de condición efectivo es $(\sigma_{\text{máx}}^2 + \alpha)/(\sigma_{\text{mín}}^2 + \alpha)$.
- Se puede ver como un multiplicador de Lagrange que impone $\|x\|_2^2 = c$: empuja a que las entradas tengan magnitud parecida en vez de grandes y opuestas que se cancelan.
- "Ridge" en estadística; en inversos es "Tikhonov".

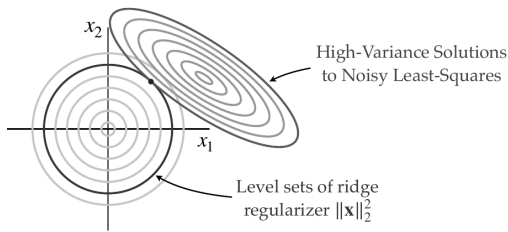


Fig. 4.12: las soluciones ruidosas (elipses azules) se arrastran hacia los círculos de $\|x\|_2^2$.

Brunton, *Optimization Bootcamp*, Cap. 4

El compromiso sesgo–varianza

- **Sin regularizar:** alta varianza. El elipsoide de soluciones posibles ante ruido es enorme y muy excéntrico.
- **Con α:** la varianza se encoge, pero la distribución se **desplaza**: aparece sesgo.
- $\alpha \to 0$: mínimos cuadrados puros. $\alpha \to \infty$: $x \to 0$.
- **Perspectiva bayesiana:** el término ridge equivale a un prior Gaussiano con media cero e igual varianza en todas las entradas (Cap. 7).

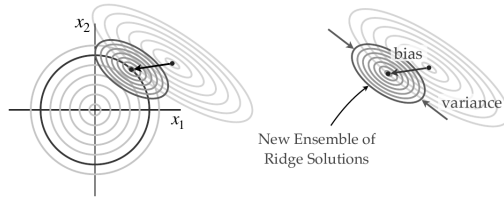


Fig. 4.13: menos varianza (elipses oscuras dentro de las claras), pero la media se desplaza.

Brunton, *Optimization Bootcamp*, Cap. 4

Ridge sin invertir: la versión SVD

Sustituyendo $A = U\Sigma V^T$ en (4.39):

$$\begin{aligned}(A^TA + \alpha I)^{-1} &= (V\Sigma^2 V^T + \alpha VIV^T)^{-1} \\ &= \left(V(\Sigma^2 + \alpha I)V^T\right)^{-1} \\ &= V(\Sigma^2 + \alpha I)^{-1}V^T.\end{aligned}$$

Lectura

Basta sumarle α a cada valor singular al cuadrado antes de invertir. Nada de matrices $n \times n$: es un filtro sobre los σ_j,

$$\frac{\sigma_j}{\sigma_j^2 + \alpha},$$

que apaga suavemente las direcciones con σ_j pequeño. Es la misma familia de *filtros espectrales* que usa el truncamiento duro de la SVD, pero continuo en vez de abrupto.

Brunton, *Optimization Bootcamp*, Cap. 4

LASSO: regularización ℓ_1

$$x = \arg\min_{x'} \ \|Ax' - b\|_2^2 + \lambda \|x'\|_1 \tag{4.40}$$

- *Least Absolute Shrinkage and Selection Operator* (Tibshirani, 1996).
- Ridge **encoge** los coeficientes; LASSO los **apaga**: devuelve un x **disperso**.
- λ se elige por **validación cruzada** (típicamente 80/20).
- Motivación: interpretabilidad. En modelos de enfermedad hay decenas de predictores y la audiencia no es técnica.
- Antecedentes: soft-thresholding de Donoho–Johnstone; garrote de Breiman; navaja de Occam.

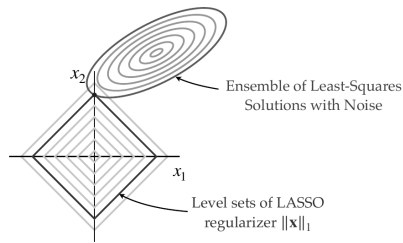


Fig. 4.14: los vértices de la bola ℓ_1 atraen la solución hacia los ejes.

Brunton, *Optimization Bootcamp*, Cap. 4

Elastic net y el mapa de las penalizaciones

Elastic net (Zou–Hastie, 2005) combina ambas:

$$x = \arg\min_{x'} \ \|Ax' - b\|_2^2 + \lambda\|x'\|_1 + \alpha\|x'\|_2^2 \quad (4.41)$$

- Ridge solo: encoge, no selecciona.
- LASSO solo: selecciona, pero con predictores muy correlacionados elige uno al azar y descarta el resto.
- Elastic net: selecciona y además **agrupa** predictores correlacionados.

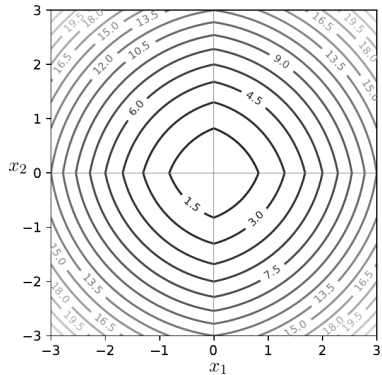


Fig. 4.15: conjuntos de nivel del elastic net con $\lambda = \alpha = 1$; se parecen a la bola $\ell_{3/2}$.

Sin solución cerrada: se resuelven con métodos proximales, coordinate descent o solvers de QP/SOCP.

Brunton, *Optimization Bootcamp*, Cap. 4

Mínimos cuadrados con restricciones de igualdad

$$\min_x \ \|Ax - b\|_2^2 \qquad \text{s.a. } Cx = d \tag{4.42}$$

Equivalentemente, como **programa cuadrático**:

$$\min_x \ \tfrac{1}{2} x^T Q x + c^T x \quad \text{s.a. } Cx = d, \qquad Q = 2A^T A, \ \ c^T = -2b^T A \tag{4.43–4.44}$$

Lagrangiano aumentado (4.45):

$$L(x, \lambda) = \tfrac{1}{2} x^T Q x + c^T x + \lambda^T (Cx - d)$$

Condiciones de optimalidad $\Rightarrow$ sistema **KKT**:

$$\begin{bmatrix} Q & C^T \\ C & 0 \end{bmatrix} \begin{bmatrix} x \\ \lambda \end{bmatrix} = \begin{bmatrix} -c \\ d \end{bmatrix} \qquad \Longleftrightarrow \qquad \begin{bmatrix} 2A^T A & C^T \\ C & 0 \end{bmatrix} \begin{bmatrix} x \\ \lambda \end{bmatrix} = \begin{bmatrix} 2A^T b \\ d \end{bmatrix} \tag{4.47–4.48}$$

Se resuelve por SVD o QR. Sin restricciones ($C = 0$) se recupera el problema estándar.

Brunton, *Optimization Bootcamp*, Cap. 4

La norma mínima como caso particular

Con $A = I$ y $b = 0$, los mínimos cuadrados restringidos se reducen a **norma mínima**:

$$\min_x \|x\|_2^2 \quad \text{s.a. } Cx = d \qquad \Longrightarrow \qquad \begin{bmatrix} I & C^T \\ C & 0 \end{bmatrix} \begin{bmatrix} x \\ \lambda \end{bmatrix} = \begin{bmatrix} 0 \\ d \end{bmatrix} \tag{4.50}$$

$$x^* = C^T (CC^T)^{-1} d$$

- Todo el capítulo es un mismo sistema KKT con distintas matrices: cambio de Q, de C o de d.
- **Sistemas con punto de silla:** la matriz KKT es simétrica pero **indefinida**, aunque Q sea definida positiva. Los solvers genéricos pueden necesitar formulaciones especiales.
- Restricciones de **desigualdad** ($Cx \leq d$, cajas, no negatividad) ya no caben en este esquema lineal: requieren QP (§4.6) o el capítulo 6.

Brunton, *Optimization Bootcamp*, Cap. 4

Qué es un programa cuadrático

$$\min_{x} \ \tfrac{1}{2} x^T Q x + c^T x \qquad \text{s.a. } Ax \leq b \tag{4.51}$$

- Q simétrica.
- $Q \succeq 0$: QP **convexo**; todo mínimo local es global.
- Q indefinida: no convexo, mucho más difícil.
- A diferencia del PL, el óptimo **no está en un vértice**: puede caer en una arista, una cara o el interior.

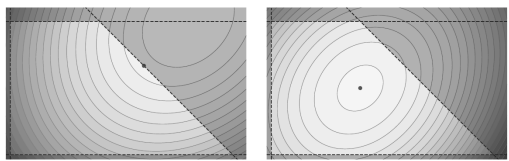


Fig. 4.16: restricciones inactivas (izq.: óptimo interior) y activas (der.: óptimo en la frontera).

Brunton, *Optimization Bootcamp*, Cap. 4

Tres familias de algoritmos

1. Conjunto activo (*active set*)

Adivinar qué restricciones están activas, resolver el subproblema de igualdades con KKT y corregir el conjunto. **El simplex de Dantzig es un método de conjunto activo.**

2. Punto interior (IPM)

Siguen una trayectoria central desde el interior del conjunto factible. **Escalan bien** a alta dimensión con restricciones lineales; son el estándar industrial para PL y QP grandes.

3. Programación cuadrática secuencial (SQP)

Para problemas **no lineales**: en cada iteración se aproxima la curvatura con el Hessiano y las restricciones por linealización, y se resuelve un QP. Convergencia **superlineal** cerca de la solución.

El QP es el ladrillo con el que se construyen los métodos modernos para problemas no lineales restringidos.

Brunton, *Optimization Bootcamp*, Cap. 4

Mínimos cuadrados *son* un QP

$$\|Ax - b\|_2^2 = (Ax - b)^T (Ax - b) = x^T A^T A x - 2b^T A x + b^T b \tag{4.52}$$

$$Q = A^T A, \qquad c^T = -2b^T A \tag{4.53}$$

- $A^T A$ es **simétrica y semidefinida positiva** para cualquier A: el QP resultante es **siempre convexo**.
- Cualquier restricción lineal (cajas, no negatividad, suma fija) se añade directamente y el problema sigue siendo un QP convexo.

En la práctica: cvxpy

Reconoce automáticamente el QP y elige un solver especializado (OSQP, SCS, ECOS, GUROBI). Para QP grandes conviene *warm starts*: `prob.solve(solver=cp.OSQP, warm_start=True)`.

Brunton, *Optimization Bootcamp*, Cap. 4

Ejemplo: portafolio de Markowitz (E4.5)

$$\min_x \ \tfrac{1}{2} x^T \Sigma x \quad \text{s.a.} \quad r^T x \geq R_0, \ \ \mathbf{1}^T x = 1, \ \ x \geq 0$$

```
x = cp.Variable(n, nonneg=True)
obj = cp.Minimize(0.5*cp.quad_form(x,
                                   Sigma))
cons = [cp.sum(x) == 1,
        r @ x >= R0]
prob = cp.Problem(obj, cons)
prob.solve(solver=cp.SCS)
```

- Σ: covarianza de los rendimientos (el riesgo).
- r: rendimientos esperados.
- $x \geq 0$: prohibición de **ventas en corto**.
- Es *el* QP canónico de las finanzas.

Brunton, *Optimization Bootcamp*, Cap. 4

Gauss–Newton: el problema

$$L(\theta) = \tfrac{1}{2} \sum_{j=1}^{m} \|y_j - N(x_j;\theta)\|_2^2 = \tfrac{1}{2} \sum_{j=1}^{m} f_j(\theta)^2 = \tfrac{1}{2} \|\mathbf{f}(\theta)\|_2^2 \tag{4.54}$$

- $\mathbf{f}(\theta)$: **vector de residuos**; f_j es el error residual en el dato j.
- θ: parámetros del modelo no lineal.
- Sin solución cerrada y **no convexo**.
- Es la formulación que cubre el **entrenamiento de redes neuronales**: N es la red y θ son pesos y sesgos (Cap. 10).
- Se ataca con Newton, pero el Hessiano exacto es caro.

Estrategia

Resolver $\nabla_\theta L = 0$ requiere un sistema no lineal de ecuaciones. **Gauss–Newton aproxima el Hessiano** usando solo primeras derivadas, de modo que cada iteración es un problema de mínimos cuadrados lineales.

Brunton, *Optimization Bootcamp*, Cap. 4

Gradiente, Jacobiano y Hessiano

$$\nabla_\theta L = \sum_{j=1}^{m} f_j(\theta) \nabla f_j(\theta) = J^T \mathbf{f} \tag{4.55}$$

$$J = D\mathbf{f} = \begin{bmatrix} \nabla f_1^T \\ \nabla f_2^T \\ \vdots \\ \nabla f_m^T \end{bmatrix} \in \mathbb{R}^{m \times n} \tag{4.56}$$

$$H = \nabla_\theta^2 L = J^T J + \sum_{j=1}^{m} f_j(\theta) \nabla^2 f_j(\theta) = J^T J + Q \tag{4.58}$$

La suposición central de Gauss–Newton

Si los residuos f_j son pequeños, el término Q (segundas derivadas) es despreciable:

$$H \approx J^T J \tag{4.59}$$

$J^T J$ es simétrica y **semidefinida positiva**: garantiza una dirección de descenso.

Brunton, *Optimization Bootcamp*, Cap. 4

La iteración de Gauss–Newton

Newton sería $\theta^{k+1} = \theta^k - H^{-1}\nabla L$, pero nunca se forma H^{-1}. Se resuelve el sistema y se da el paso:

$$J_k^T J_k \, s^k = -J_k^T \mathbf{f}(\theta^k) \qquad (4.64a) \qquad\qquad \theta^{k+1} = \theta^k + s^k \qquad (4.64b)$$

Equivalencia con un mínimos cuadrados

Resolver (4.64a) equivale exactamente a

$$\min_{s} \ \tfrac{1}{2}\|Js + \mathbf{f}\|_2^2 \qquad (4.66)$$

Cada iteración de Gauss–Newton es un problema de mínimos cuadrados lineales — el del capítulo, con $A = J$ y $b = -\mathbf{f}$.

Consecuencia: todo lo anterior se reutiliza dentro de Gauss–Newton (SVD, QR, regularización, gradiente conjugado).

Brunton, *Optimization Bootcamp*, Cap. 4

Los modos de falla

Cuándo Gauss–Newton se rompe

- **Residuos grandes:** la aproximación $H \approx J^T J$ ignora términos de segundo orden que ya no son pequeños $\Rightarrow$ **divergencia lejos de la solución**.
- $J^T J$ **singular o casi singular:**
  1. el paso s^k puede ser enorme y *pasarse* de la solución (overshoot);
  2. el sistema lineal está mal condicionado: pequeños errores en J o $\mathbf{f}$ producen pasos muy distintos;
  3. la dirección puede no ser de descenso y la iteración se *estanca*.
- Salvo residuos muy pequeños, la convergencia es solo **de primer orden**, no cuadrática.

Vasta literatura dedicada a estas patologías. La cura práctica estándar es **amortiguar** el Hessiano.

Brunton, *Optimization Bootcamp*, Cap. 4

Levenberg–Marquardt: amortiguar el Hessiano

$$H \approx J^T J + \alpha I \qquad (4.73) \qquad \Longrightarrow \qquad (J^T J + \alpha I)\,\Delta\theta = -J^T \mathbf{f} \qquad (4.74)$$

- $\alpha = 0$: **Gauss–Newton** puro (rápido cerca del óptimo).
- $\alpha \to \infty$: $\Delta\theta = -\frac{1}{\alpha} J^T \mathbf{f}$: **descenso de gradiente** con paso $1/\alpha$ (lento pero seguro).
- α **interpola** entre exploración y explotación.

Dos lecturas

1. Es **regularización de Tikhonov** (§4.4) aplicada al paso de Newton: suma α a los valores singulares al cuadrado.

2. Es una **región de confianza**: α controla cuánto se confía en el modelo cuadrático local.

Usado en visión por computadora, ajuste de curvas, robótica y estimación de parámetros en control. Es el "mínimos cuadrados no lineales" por defecto.

Brunton, *Optimization Bootcamp*, Cap. 4

Cómo se adapta α

- Comparar la reducción **real** de la pérdida con la **predicha** por el modelo cuadrático:

$$\rho = \frac{\Delta L_{\text{actual}}}{\Delta L_{\text{predicho}}} \tag{4.76}$$

$$\Delta L_{\text{actual}} = L(\theta^k) - L(\theta^{k+1}) \tag{4.77}$$

$$\Delta L_{\text{predicho}} = -J^T \mathbf{f}\, \Delta\theta - \Delta\theta^T (J^T J + \alpha I)\Delta\theta \tag{4.78}$$

- Regla típica: $\rho > 0{,}75 \Rightarrow$ reducir α; $\rho < 0{,}25 \Rightarrow$ aumentar α. Multiplicar o dividir por un factor $\nu > 1$.
- Se usa el cociente, no la magnitud absoluta: cerca del mínimo las reducciones son diminutas pero deben seguir siendo las predichas.
- Inicialización habitual: $\alpha \sim 10^{-3}$.
- Variante más fina: escalar cada entrada diagonal, $H = J^T J + \alpha\, \mathrm{diag}(J^T J)$ (4.80), para respetar la **curvatura local** de cada parámetro.

Brunton, *Optimization Bootcamp*, Cap. 4

Gradiente conjugado: para qué

$$Ax = b, \qquad A \in \mathbb{R}^{n \times n} \text{ simétrica definida positiva} \tag{4.81}$$

- **No factoriza** A: solo necesita productos matriz–vector.
- Ideal para sistemas **grandes y dispersos**; el ejemplo clásico es la ecuación de Poisson en dinámica de fluidos.
- Convergencia garantizada en **a lo sumo** n **iteraciones**.
- Su uso estrella aquí: resolver la ecuación de Gauss–Newton

$$J^T J \, \Delta x = -J^T \mathbf{f} \tag{4.82}$$

  sin formar ni almacenar $J^T J$. La memoria se dispara: J es $m \times n$ y $J^T J$ sería $n \times n$ densa.
- También mejora la estabilidad frente a la inversión explícita de un Hessiano mal condicionado. El precio: la convergencia sigue dependiendo de $\kappa(J^T J)$.

Brunton, *Optimization Bootcamp*, Cap. 4

La idea: base A-conjugada

Direcciones **conjugadas** = ortogonales en el producto interior $\langle u, v \rangle_A = u^T A v$:

$$p_i^T A p_j = 0 \qquad \forall\, i \neq j \tag{4.83}$$

Si $A \succ 0$, el conjunto $\{p_j\}_{j=1}^n$ es una **base de** $\mathbb{R}^n$. Se expande la solución y se despeja cada coeficiente:

$$x = \sum_{j=1}^{n} \alpha_j p_j \tag{4.84}$$

multiplicando $Ax = b$ por p_k^T y usando la conjugación, todos los términos salvo uno se anulan:

$$\alpha_k = \frac{p_k^T b}{p_k^T A p_k} \tag{4.88}$$

Por qué es mejor que el descenso de gradiente

En CG, minimizar a lo largo de una dirección **no destruye** el progreso hecho en las anteriores. El descenso de gradiente sí lo destruye: por eso zigzaguea.

Brunton, *Optimization Bootcamp*, Cap. 4

El algoritmo

Inicializar e iterar:

$$x_0, \qquad r_0 = b - Ax_0, \qquad p_0 = r_0 \tag{4.94}$$

$$\alpha_k = \frac{r_k^T r_k}{p_k^T A p_k} \tag{4.95a} \qquad x^{k+1} = x^k + \alpha_k p_k \tag{4.95b} \qquad r_{k+1} = r_k - \alpha_k A p_k \tag{4.95c}$$

$$\beta_k = \frac{r_{k+1}^T r_{k+1}}{r_k^T r_k} \tag{4.96a} \qquad p_{k+1} = r_{k+1} + \beta_k p_k \tag{4.96b}$$

Qué guarda: solo tres vectores (x^k, r_k, p_k). Memoria $\Theta(n)$.

Qué cuesta por iteración: un producto Ap_k, dos productos internos y tres operaciones vectoriales. Nada de factorizar.

Brunton, *Optimization Bootcamp*, Cap. 4

Convergencia, precondicionamiento y GMRES

$$\|x^k - x^*\|_A \leq C \left(\frac{\kappa(A) - 1}{\kappa(A) + 1} \right)^k \tag{4.97}$$

- Cota en la **norma de energía** $\|\cdot\|_A$; κ es el cociente de valores propios extremos.
- Con κ grande la cota es casi 1 y CG se arrastra: **el precondicionamiento es obligatorio en la práctica**.
- Se resuelve $M^{-1}Ax = M^{-1}b$ (4.98) con $M \approx A$ pero fácil de invertir.

Panel 4.4: GMRES

Cuando A **no** es simétrica definida positiva, CG no aplica. **GMRES** minimiza $\|r\|_2$ sobre el subespacio de Krylov usando la iteración de Arnoldi. El costo de ortogonalización crece con las iteraciones, así que se usa reiniciado (**GMRES(m)**). También depende críticamente del precondicionador. Es el *caballo de batalla* para sistemas no simétricos (fluidos, electromagnetismo).

Brunton, *Optimization Bootcamp*, Cap. 4

Puntos para recordar

1. **Dos regímenes, una matriz:** sobredeterminado $\rightarrow$ mínimos cuadrados; subdeterminado $\rightarrow$ norma mínima. Ambas son $A^{\dagger}b$.
2. **Nunca $A^{T}A$:** se usa la SVD. Es estable, siempre existe y da la aproximación óptima de bajo rango (Eckart–Young).
3. **Conditioning es el enemigo:** el ruido se multiplica por $\kappa(A)$. Truncar la SVD o regularizar (ridge) son las dos curas.
4. **ℓ_1 frente a ℓ_2:** ℓ_1 da robustez a outliers (mediana) y dispersidad (LASSO, compressed sensing). El precio es perder la fórmula cerrada.
5. **Las restricciones son un sistema KKT:** una sola estructura matricial cubre norma mínima y mínimos cuadrados restringidos.
6. **Mínimos cuadrados es un QP convexo:** cualquier restricción lineal encaja; `cvxpy` lo resuelve.
7. **No lineal:** Gauss–Newton convierte cada iteración en un mínimos cuadrados; Levenberg–Marquardt lo amortigua para no divergir.
8. **Grande y disperso:** gradiente conjugado, con productos matriz–vector y memoria $\Theta(n)$.

Brunton, *Optimization Bootcamp*, Cap. 4

El mapa completo

Herramienta	**Qué problema resuelve**
Ecuaciones normales	LS sobredeterminado, rango completo, bien condicionado
SVD / pseudo-inversa	LS y norma mínima, estable, cualquier rango
Truncamiento SVD	conditioning malo, ruido en los modos débiles
Ridge (Tikhonov)	conditioning malo con sesgo controlado
LASSO / elastic net	selección de variables, modelos interpretables
KKT	restricciones de igualdad
QP (active set / IPM)	restricciones de desigualdad, cajas, portafolios
Gauss–Newton	regresión no lineal, redes neuronales
Levenberg–Marquardt	Gauss–Newton con residuos grandes o mal condicionado
Gradiente conjugado	sistemas enormes y dispersos, sin factorizar

Todo el capítulo es una sola idea con distintos disfraces: proyectar b sobre un subespacio y controlar cuánto se confía en las direcciones débiles.

Para profundizar

Libros

- Brunton & Kutz, *Data-Driven Science and Engineering* (2022).
- James, Witten, Hastie & Tibshirani, *An Introduction to Statistical Learning* (2013).
- Trefethen, *Numerical Linear Algebra* (2022).
- Antoulas, *Approximation of Large-Scale Dynamical Systems* (2005).

Artículos fundacionales

- Tikhonov (1943), *On the stability of inverse problems*.
- Tibshirani (1996), *Regression shrinkage and selection via the Lasso*.
- Zou & Hastie (2005), *Regularization and variable selection via the elastic net*.
- Donoho (2006), *Compressed sensing*.
- Candès, Romberg & Tao (2006), *Stable signal recovery from incomplete and inaccurate measurements*.

Siguiente capítulo: optimización no suave y global.